Revisão para a P1 – Programação para Dispositivos Móveis I

Leia cada enunciado e, quando houver código, observe os nomes, parâmetros e chamadas. Marque apenas uma alternativa.

1. Observe o código:

```
@Preview
@Composable
fun PreviewTitulo() {
 Titulo()
}
```

Qual é a finalidade dessa função?

A) Executar a navegação para a tela Titulo.
B) Exibir uma prévia do Composable Titulo no editor.
C) Gravar o texto Titulo nas preferências do aplicativo.
D) Criar uma Activity chamada Titulo.
E) Definir o conteúdo principal da aplicação.

2. Uma função de interface precisa receber uma lista para exibir os produtos cadastrados. Qual declaração permite receber essa lista como parâmetro?

A) fun ListaProdutos(navController: NavController)
B) fun ListaProdutos(nome: String)
C) fun ListaProdutos(produtos: List<Produto>)
D) fun ListaProdutos()
E) fun ListaProdutos(chave: Preferences.Key<String>)

3. Um formulário deve avisar à tela que mantém a lista quando um novo produto for confirmado. Qual opção permite enviar essa ação para a tela responsável?

A) Receber uma função de callback como parâmetro.
B) Declarar uma rota chamada cadastro.
C) Criar uma função @Preview para o formulário.
D) Declarar uma chave com stringPreferencesKey.
E) Chamar collectAsState no botão.

4. Considere o trecho:

```
Button(onClick = {
 navController.navigate("cadastro")
}) {
 Text("Novo produto")
}
```

Qual destino será solicitado ao acionar o botão?

A) O destino inicial configurado no NavHost.
B) A tela que contém o Preview.
C) A tela de lista, independentemente da rota.
D) O destino associado à rota cadastro.
E) A tela que contém o DataStore.

5. Considere a configuração:

```
NavHost(
 navController = navController,
 startDestination = "lista"
```

) {

composable("lista") { TelaListaCompras(navController) }

composable("cadastro") { TelaCadastroProduto(navController) } }

Qual alternativa descreve uma consequência dessa configuração?

A) O DataStore passa a iniciar com uma lista vazia.

B) O botão de cadastro é criado automaticamente.

C) A rota lista é definida como destino inicial e as duas rotas são registradas

D) A aplicação passa a ter duas Activities.

E) A rota cadastro será aberta primeiro.

6. A aplicação tenta navegar para detalhes, mas essa rota não aparece na configuração do NavHost. Qual alteração corresponde ao problema apresentado?

A) Trocar stringPreferencesKey por longPreferencesKey.

B) Adicionar ao NavHost um destino registrado com a rota detalhes.

C) Alterar o texto exibido pelo botão.

D) Colocar @Preview na função do botão.

E) Mudar o tipo de List<Produto> para String.

7. Considere a declaração:

val NOME_USUARIO = stringPreferencesKey("nome_usuario")

Qual é a finalidade de NOME_USUARIO?

A) Guardar diretamente o texto digitado em uma variável

comum.

B) Indicar qual tela será aberta após salvar.

C) Representar a lista de produtos cadastrados.

D) Definir a função que atualiza a interface.

E) Identificar a preferência usada para armazenar um valor textual.

8. Analise o trecho:

```
context.dataStore.edit { preferences ->
 preferences[NOME_USUARIO] = nome
}
```

O que esse bloco faz?

A) Atualiza o valor da preferência identificada por NOME_USUARIO com o conteúdo de nome.

B) Recupera o valor salvo e o exibe em um Text.

C) Registra uma rota de navegação chamada nome.

D) Cria um estado temporário para o campo de texto.

E) Converte o nome em uma lista de produtos.

9. Considere a expressão:

```
preferences[NOME_USUARIO] ?: ""
```

Se ainda não houver valor salvo para essa chave, qual resultado será utilizado?

A) A rota lista.

B) O texto nome_usuario.

C) Uma lista vazia.

D) Uma String vazia.

E) Um valor numérico igual a zero.

10. Considere o código:

val nomeSalvo by nomeFlow.collectAsState(initial = "")

Qual é o papel de collectAsState nessa expressão?

A) Criar uma chave de preferência para salvar o nome.
B) Registrar o destino inicial da navegação.
C) Disponibilizar os valores do Flow como estado observado pelo Compose.
D) Alterar o valor armazenado no DataStore.
E) Criar uma nova função Composable.

11. Uma tela foi dividida em três Composables: formulário, lista e resumo. Qual característica é favorecida por essa divisão?

A) Cada parte passa a armazenar dados permanentemente.
B) A interface fica organizada em partes que podem ser mantidas e reutilizadas.
C) A aplicação deixa de precisar de parâmetros.
D) Cada parte passa a funcionar como uma Activity independente.
E) A navegação entre telas deixa de precisar de rotas.

12. Uma função é declarada como fun ListaProdutos(produtos: List<Produto>). Qual chamada fornece um argumento compatível com esse parâmetro?

A) ListaProdutos("Arroz")

B) ListaProdutos(navController)

C) ListaProdutos(NOME_USUARIO)

D) ListaProdutos()

E) ListaProdutos(produtos) quando produtos é uma lista de Produto.

13. A variável nomeSalvo recebe os valores de nomeFlow por meio de collectAsState. Qual afirmação distingue esse trecho do bloco de gravação com edit?

A) collectAsState registra uma rota, enquanto edit registra uma tela.

B) Os dois blocos servem apenas para alterar o texto de um botão.

C) collectAsState observa valores como estado; edit modifica as preferências armazenadas.

D) edit serve para abrir o Preview; collectAsState cria uma Activity.

E) collectAsState cria uma chave; edit declara o tipo String.

14. Um botão contém navController.navigate("cadastro"). Para que a chamada encontre um destino correspondente, o que precisa existir na configuração de navegação?

A) Um composable("cadastro") dentro do NavHost.

B) Uma chave stringPreferencesKey("cadastro").

C) Uma função PreviewCadastro() obrigatoriamente.

D) Uma variável nomeSalvo inicializada com texto.

E) Uma lista chamada cadastro.

15. O valor de uma preferência textual precisa ser lido para aparecer na interface e também atualizado quando o usuário salvar outro nome. Qual associação está correta?

A) navigate lê; NavHost grava.
B) @Preview lê; composable grava.
C) rememberNavController lê; startDestination grava.
D) data fornece o fluxo de leitura; edit altera as preferências.
E) stringPreferencesKey lê; collectAsState grava.